DESARROLLO FULL STACK · UNIDAD 3

Aplicaciones en tiempo real con WebSockets

Integración de bibliotecas de WebSockets en un proyecto de back-end

Facultad de Ingeniería de Sistemas · Sesión teórico-práctica de 90 minutos

RUTA DE LA SESIÓN

90 minutos en cinco momentos

Momento	Minutos	Qué haremos
Inicio	0 – 8	Pregunta detonante y logro de la sesión
Utilidad	8 – 18	WebSockets en la vida real: WhatsApp Web, Discord, Slack, Trello, Binance…
Transformación	18 – 60	Qué es, handshake, ciclo de vida, bibliotecas, 6 ejemplos ejecutados y errores comunes
Práctica	60 – 82	Laboratorio: el Slack de la clase, todos conectados al mismo chat
Cierre	82 – 90	Metacognición y puente a JWT

INICIO · PREGUNTA DETONANTE

¿Cómo aparece un mensaje en WhatsApp Web sin que presiones F5?

Lluvia de ideas en parejas · 3 minutos · anota tu hipótesis en una frase

LOGRO DE LA SESIÓN

Al finalizar, integras una biblioteca de WebSockets en un back-end Node.js y conectas un front-end que recibe cambios de datos en tiempo real.

01

Explica

por qué HTTP no basta y cómo funciona el handshake WebSocket

02

Implementa

un servidor con ws y con Socket.IO: eventos, salas y acks

03

Integra

el canal en tiempo real con la API REST (CRUD) y el front-end

UTILIDAD

El tiempo real ya está en las apps que usas a diario

Chat y mensajería

WhatsApp Web, Slack, chats de soporte

Dashboards en vivo

ventas, monitoreo de servidores, bolsa

Seguimiento GPS

el repartidor de tu delivery moviéndose en el mapa

Edición colaborativa

Google Docs, Figma, pizarras compartidas

Juegos multijugador

posición de jugadores, marcadores, Kahoot

Notificaciones

“te llegó un pago”, “tu pedido cambió de estado”

WEBSOCKETS EN LA VIDA REAL

Si hoy abriste WhatsApp Web, Slack o Discord, ya usaste WebSockets

Ninguna de estas apps recarga la página para mostrarte algo nuevo: mantienen abierta una conexión wss:// y el servidor empuja cada novedad en cuanto ocurre.

VIDA REAL · MENSAJERÍA Y COLABORACIÓN

Los productos más conocidos que viven sobre WebSockets

Producto	Para qué usa WebSockets	Detalle técnico
WhatsApp Web	Recibir y enviar mensajes en el navegador sin recargar	Conexión segura a servidores wss://...web.whatsapp.com/ws
Discord	Mensajes, presencia y eventos de cada servidor	La Gateway es una conexión WebSocket segura y persistente
Slack	Las apps y bots reciben eventos del espacio de trabajo	Socket Mode: una URL WebSocket en vez de un endpoint HTTP público
Trello	Los cambios de otros usuarios aparecen en tu tablero al instante	Versión modificada de Socket.IO, con polling como respaldo
Figma	Edición multijugador: cursores y cambios de todos en vivo	Cada cliente sincroniza el documento con el servidor por WebSocket

VIDA REAL · FINANZAS, STREAMING Y JUEGOS

Donde cada milisegundo cuenta

Binance

Precios, operaciones y libro de órdenes en vivo para quien programa bots de trading

stream.binance.com

Twitch

El chat de cada transmisión en vivo; los bots del chat se conectan por el mismo canal

irc-ws.chat.twitch.tv

Lichess

Cada jugada de ajedrez llega al rival al instante; un servidor dedicado solo a WebSockets

lila-ws + Redis

Compruébalo tú mismo: abre web.whatsapp.com o discord.com → F12 → pestaña Network → filtro **WS** → clic en la conexión → pestaña **Messages**.

EL PROBLEMA

En HTTP el servidor nunca habla primero

HTTP + polling

C → S GET /mensajes

S → C 200 [] (nada nuevo)

C → S GET /mensajes

S → C 200 [] (nada nuevo)

C → S GET /mensajes

S → C 200 [“Hola”]

Cada pregunta abre una petición con cabeceras completas. Retraso = intervalo de consulta.

WebSocket

C → S GET / Upgrade: websocket

S → C 101 Switching Protocols

C ⇄ S canal abierto

S → C “Hola” (cuando ocurre)

C → S “¿Cómo estás?”

S → C “Bien”

Un solo handshake; luego ambos lados envían mensajes cuando quieren.

DEFINICIÓN

WebSocket es un protocolo que deja abierto un canal entre navegador y servidor para que ambos envíen mensajes cuando quieran

HTTP es como enviar cartas

Cada pregunta va en un sobre nuevo con remitente y dirección completos. El servidor solo contesta cartas: nunca escribe por iniciativa propia.

WebSocket es una llamada

Marcas una sola vez. Mientras nadie cuelgue, cualquiera de los dos habla cuando quiere y el otro escucha al instante.

1. Empieza como HTTP

2. Pide cambiar de protocolo

3. Termina cuando alguien cierra

ALTERNATIVAS

Cuatro formas de “sentir” tiempo real

Técnica	Cómo funciona	Dirección	Latencia	Uso típico
Polling	El cliente pregunta cada N segundos	C → S	Alta (= N)	Datos que cambian poco
Long polling	El servidor retiene la respuesta hasta que hay algo	C → S	Media	Respaldo en redes viejas
SSE	Flujo HTTP abierto, solo de servidor a cliente	S → C	Baja	Noticias, avisos, progreso
WebSocket	Conexión TCP persistente y bidireccional	C ⇄ S	Muy baja	Chat, juegos, colaboración

Regla práctica: si el cliente también necesita enviar eventos frecuentes, elige WebSocket; si solo escucha, SSE puede bastar.

CÓMO EMPIEZA · RFC 6455

El handshake: HTTP pide “cambiar de protocolo”

Petición del cliente

```
GET /chat HTTP/1.1
Host: localhost:8080
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Respuesta del servidor

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

Tras el **101** ya no hay HTTP: viajan **frames** de texto o binario en ambos sentidos sobre la misma conexión TCP. Usa los puertos 80/443, así que atraviesa firewalls y proxies.

CONCEPTOS CLAVE

Cuatro ideas que debes dominar

Full-dúplex

cliente y servidor envían a la vez, sin turnos

Persistente

la conexión vive hasta que alguien la cierra

Frames

mensajes de texto (JSON) o binarios con cabecera mínima

ws:// y wss://

como http y https; en producción siempre wss://

Estados de la conexión · socket.readyState

0 CONNECTING	1 OPEN	2 CLOSING	3 CLOSED

CICLO DE VIDA DE UNA CONEXIÓN

Cuatro eventos gobiernan todo lo que pasa

new WebSocket()

Inicia el handshake. Estado: CONNECTING

→

open

El servidor aceptó (101). Ya puedes usar send()

→

message

Se dispara una vez por cada frame recibido, todas las veces que haga falta

→

close

Alguien cerró o se cayó la red. Trae un código: 1000 = normal

⚠ **error** puede aparecer en cualquier momento (servidor apagado, red caída) y casi siempre va seguido de **close**. Ahí decides si reintentar.

Son los mismos eventos en el navegador y en Node 22 (API nativa). En la biblioteca **ws** se escuchan con .on('message'); en Socket.IO cada evento tiene el nombre que tú elijas.

EJEMPLO 5 · API NATIVA · EJ5/CLIENTE-NATIVO.JS

El ciclo de vida, impreso paso a paso

```
const ws = new WebSocket('ws://localhost:8080');
const E = ['CONNECTING', 'OPEN', 'CLOSING', 'CLOSED'];
const estado = (paso) =>
  console.log(paso.padEnd(13), '->', E[ws.readyState]);
estado('al crear');
ws.onopen = () => {
  estado('onopen');
  ws.send('Hola desde la API nativa');
};
ws.onmessage = (e) =>
  console.log('onmessage     ->', e.data);
ws.onerror = () => estado('onerror');
ws.onclose = (e) => {
  estado('onclose');
  console.log('código de cierre:', e.code);
};
setTimeout(() => {
  ws.close(1000, 'fin de la demo');
  estado('tras close()');
}, 300);
```

salida real (servidor del ej. 1)

```
al crear      -> CONNECTING
onopen        -> OPEN
onmessage     -> Bienvenido al servidor WebSocket
onmessage     -> eco: Hola desde la API nativa
tras close()  -> CLOSING
onclose       -> CLOSED
código de cierre: 1000
```

Fíjate: justo después de **new** el estado es CONNECTING y no se puede enviar todavía. close() no es instantáneo: pasa por CLOSING antes de CLOSED.

BIBLIOTECAS PARA EL BACK-END

No se programa el protocolo: se integra una biblioteca

Biblioteca	Stack	Qué aporta	Cuándo elegirla
ws	Node.js	WebSocket estándar, liviano y rápido	Control total, clientes nativos
Socket.IO	Node.js	Eventos con nombre, salas, acks, reconexión automática	Chats y apps de producto
Spring WebSocket + STOMP	Java	Integración con Spring Boot y tópicos pub/sub	Back-ends Java empresariales
Django Channels / FastAPI	Python	WebSockets sobre ASGI	Back-ends Python
WebSocket API	Navegador	Cliente nativo: new WebSocket(url)	Front-end sin dependencias

Hoy usamos **ws** para entender el protocolo y **Socket.IO** para construir la aplicación.

EJEMPLO 1 · BIBLIOTECA WS

Servidor WebSocket con eco y broadcast

```
// ej1/server.js
const { WebSocketServer, WebSocket } = require('ws');
const wss = new WebSocketServer({ port: 8080 });

wss.on('connection', (socket) => {
 socket.send('Bienvenido al servidor WebSocket');

 socket.on('message', (data) => {
  const texto = data.toString();
  // broadcast: reenviar a todos
  wss.clients.forEach((c) => {
   if (c.readyState === WebSocket.OPEN)
    c.send(`eco: ${texto}`);
  });
 });

 socket.on('close', () => console.log('desconectado'));
});
```

```
$ npm i ws
```

connection

nace un socket por cada cliente

message

llega un frame; viene como Buffer, por eso toString()

wss.clients

conjunto de conexiones vivas: base del broadcast

EJEMPLO 1 · CLIENTE Y RESULTADO

Un cliente Node se conecta, saluda y recibe el eco

```
// ej1/client.js
const WebSocket = require('ws');
const ws = new WebSocket('ws://localhost:8080');

ws.on('open', () => {
  console.log('[cliente] conexión abierta');
  ws.send('Hola desde Node');
});
ws.on('message', (m) =>
  console.log(`[cliente] llegó: ${m}`));
setTimeout(() => ws.close(), 500);
```

salida real · 2 terminales

```
$ node ej1/server.js
[server] escuchando en ws://localhost:8080
[server] cliente conectado. Total: 1
[server] recibido: Hola desde Node
[server] cliente desconectado. Total: 0
$ node ej1/client.js
[cliente] conexión abierta
[cliente] llegó: Bienvenido al servidor WebSocket
[cliente] llegó: eco: Hola desde Node
```

Observa el orden: primero el mensaje de bienvenida (lo envía el servidor sin que nadie lo pida) y luego el eco.

EJEMPLO 6 · WS + PROTOCOLO JSON · EJ6/SERVER.JS

ws solo transporta texto: el “idioma” lo defines tú

```
const wss = new WebSocketServer({ port: 8081 });
const responder = (ws, tipo, datos) =>
  ws.send(JSON.stringify({ tipo, datos }));

const manejadores = {                  // tipo -> función
  ping:  (ws)    => responder(ws, 'pong'),
  sumar: (ws, d) => responder(ws, 'resultado', d.a + d.b),
};

wss.on('connection', (ws) => {
  ws.on('message', (raw) => {
    let msg;
    try { msg = JSON.parse(raw); }
    catch { return responder(ws, 'error', 'JSON inválido'); }
    const fn = manejadores[msg.tipo];
    if (!fn)
      return responder(ws, 'error', `tipo desconocido: ${msg.tipo}`);
    fn(ws, msg.datos);
  });
});
```

Sobre { tipo, datos }

todo mensaje dice qué es y qué trae

Router

un objeto tipo → función reemplaza el if/else gigante

Defensa

try/catch y tipo desconocido: el servidor nunca se cae por un mensaje malo

EJEMPLO 6 · RESULTADO

Cuatro mensajes enviados, cuatro respuestas distintas

salida real · -> enviado por el cliente · <- respuesta del servidor

```
$ node ej6/client.js
-> {"tipo":"ping"}
-> {"tipo":"sumar","datos":{"a":7,"b":5}}
-> {"tipo":"borrarTodo"}
-> esto no es JSON
<- {"tipo":"pong"}
<- {"tipo":"resultado","datos":12}
<- {"tipo":"error","datos":"tipo desconocido: borrarTodo"}
<- {"tipo":"error","datos":"JSON inválido"}
```

ping
tipo válido sin datos

sumar
7 + 5 = 12 en el servidor

borrarTodo
no existe: se rechaza

texto suelto
no es JSON: error, sin caerse

SOCKET.IO POR DENTRO

Lo que Socket.IO hace por ti sin que lo veas

01

Conecta seguro

Su capa de transporte (Engine.IO) abre primero con HTTP long-polling, que funciona incluso detrás de proxies estrictos

02

Sube a WebSocket

Si la red lo permite, hace el upgrade a WebSocket; si no, sigue con polling y tu código no cambia

03

Late el corazón

Envía ping/pong periódicos (heartbeat) para detectar conexiones muertas

04

Se reconecta

Si se corta, reintenta solo y guarda en cola los emit hechos mientras estaba desconectado

Eventos reservados: `connection` `disconnect` `connect_error` no los uses como nombres propios

SOCKET.IO EN UNA TABLA

¿A quién le llega cada emit?

Instrucción	Destinatarios
socket.emit('ev', datos)	Solo a ese cliente
io.emit('ev', datos)	A todos los clientes conectados
socket.broadcast.emit('ev', datos)	A todos menos al emisor
io.to('sala').emit('ev', datos)	A todos los de la sala
socket.to('sala').emit('ev', datos)	A la sala, menos al emisor
socket.join('sala')	El cliente entra a una sala

Ojo: Socket.IO usa su propio protocolo sobre WebSocket. Un cliente **ws** puro no puede hablar con un servidor Socket.IO: usa **socket.io-client**.

EJEMPLO 2 · SOCKET.IO

Chat con eventos con nombre, salas y acuse de recibo

```
// ej2/server.js
const { Server } = require('socket.io');
const io = new Server(3000, { cors: { origin: '*' } });

io.on('connection', (socket) => {
  socket.on('sala:unirse', ({ sala, usuario }) => {
    socket.join(sala);
    socket.data.usuario = usuario;
    socket.to(sala).emit('sistema', `${usuario} se unió a #${sala}`);
  });

  socket.on('chat:mensaje', ({ sala, texto }, ack) => {
    const msg = { de: socket.data.usuario, texto };
    io.to(sala).emit('chat:mensaje', msg); // toda la sala
    ack({ ok: true });                     // acuse
  });
});
```

```
$ npm i socket.io
```

Eventos

nombres tipo dominio:acción y objetos JSON automáticos

Salas

join() agrupa sockets; to() limita el envío

ack

callback que confirma al emisor

EJEMPLO 2 · CLIENTES Y RESULTADO

Ana y Luis conversan en la sala #fullstack

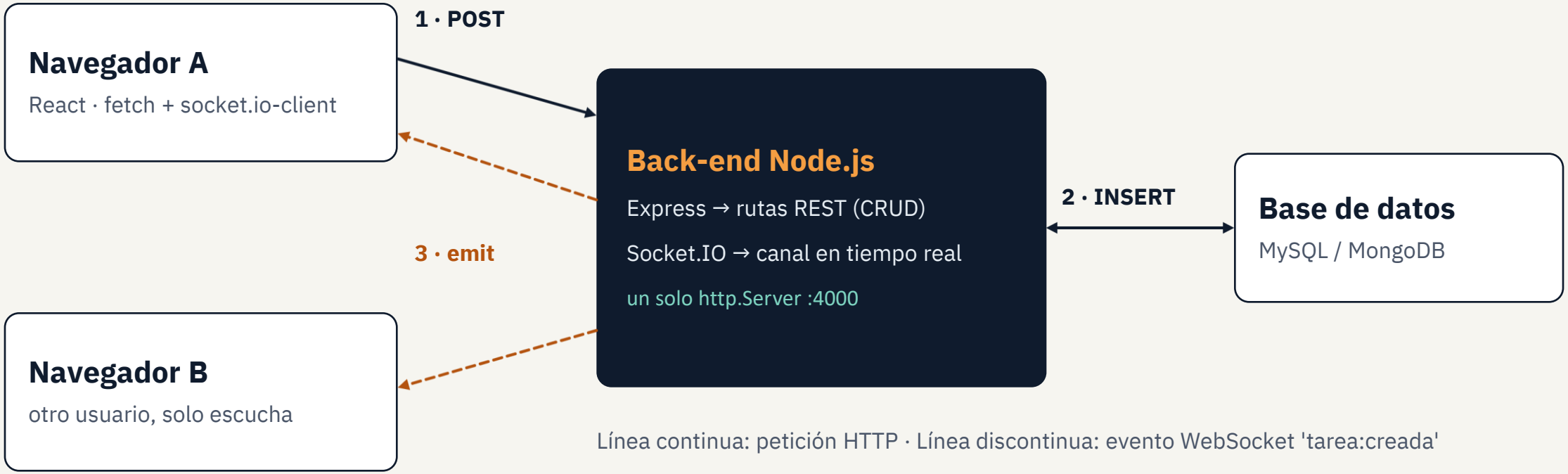
```
// ej2/clients.js (resumen)
const { io } = require('socket.io-client');
const crear = (usuario) => {
  const s = io('http://localhost:3000');
  s.on('sistema', (t) =>
    console.log(`[${usuario}] (sistema) ${t}`));
  s.on('chat:mensaje', (m) =>
    console.log(`[${usuario}] ${m.de}: ${m.texto}`));
  s.emit('sala:unirse', { sala: 'fullstack', usuario });
  return s;
};
const ana = crear('Ana');
const luis = crear('Luis');
ana.emit('chat:mensaje',
  { sala: 'fullstack', texto: '¿Ya probaron WebSockets?' },
  (r) => console.log('[Ana] ack del servidor:', r));
```

salida real

```
$ node ej2/clients.js
[Ana] (sistema) Luis se unió a #fullstack
[Ana] Ana: ¿Ya probaron WebSockets?
[Luis] Ana: ¿Ya probaron WebSockets?
[Ana] ack del servidor: { ok: true }
[Ana] Luis: ¡Sí, en tiempo real!
[Luis] Luis: ¡Sí, en tiempo real!
servidor:
[server] Ana salió
[server] Luis salió
```

INTEGRACIÓN CON EL PROYECTO

REST cambia los datos; WebSocket avisa a todos



Línea continua: petición HTTP · Línea discontinua: evento WebSocket 'tarea:creada'

PASO A PASO

¿Qué ocurre cuando Ana crea una tarea y Luis la ve aparecer?

1 · navegador de Ana

Pulsa “Agregar”: React hace **fetch POST /api/tareas**

2 · Express

Valida el body y ejecuta el **INSERT** en la base de datos

3 · base de datos

Confirma y devuelve la tarea con su **id** definitivo

4 · Socket.IO

io.emit('tarea:creada', tarea) a todos los sockets conectados

5 · Express

Responde **201 Created** a Ana con la tarea en JSON

6 · navegador de Luis

useTareas recibe el evento, llama **setTareas** y React vuelve a pintar

Luis nunca hizo una petición: el dato le llegó empujado. Las tarjetas oscuras son la parte en tiempo real que agregamos hoy.

EJEMPLO 3 · EXPRESS + SOCKET.IO

Cada operación CRUD emite un evento

```
const app = express();
app.use(express.json());
const server = http.createServer(app);  // un solo servidor
const io = new Server(server, { cors: { origin: '*' } });

app.post('/api/tareas', (req, res) => {
  const t = { id: nextId++, titulo: req.body.titulo, hecha: false };
  tareas.push(t);           // en el proyecto: INSERT en la BD
  io.emit('tarea:creada', t);  // avisa a todos
  res.status(201).json(t);
});

app.delete('/api/tareas/:id', (req, res) => {
  tareas = tareas.filter(x => x.id != req.params.id);
  io.emit('tarea:eliminada', { id: Number(req.params.id) });
  res.status(204).end();
});
server.listen(4000);  // ¡server.listen, no app.listen!
```

```
$ npm i express socket.io
```

Mismo puerto

Express y Socket.IO comparten el http.Server

Emitir después

solo si la operación en la BD fue exitosa

PUT

mismo patrón: emite tarea:actualizada

EJEMPLO 3 · RESULTADO

El dashboard se entera de cada cambio antes de que REST responda

salida real · un socket “dashboard” escucha mientras otro cliente llama a la API

```
$ node ej3/demo.js
[dashboard] evento tarea:creada -> {"id":1,"titulo":"Diseñar BD","hecha":false}
[REST] POST /api/tareas -> 201 {"id":1,"titulo":"Diseñar BD","hecha":false}
[dashboard] evento tarea:actualizada -> {"id":1,"titulo":"Diseñar BD","hecha":true}
[REST] PUT /api/tareas/1 -> 200
[dashboard] evento tarea:eliminada -> {"id":1}
[REST] DELETE /api/tareas/1 -> 204
```

Códigos HTTP correctos

201 al crear, 200 al actualizar, 204 al eliminar

Evento por operación

creada, actualizada y eliminada llevan los datos justos

Sin recargar

el dashboard nunca hizo un GET y está al día

FRONT-END REACT · SRC/HOOKS/USETAREAS.JS

REST para cargar, WebSocket para mantenerse al día

```
import { useEffect, useState } from 'react';
import { io } from 'socket.io-client';
const socket = io('http://localhost:4000');
export function useTareas() {
  const [tareas, setTareas] = useState([]);
  useEffect(() => {
    fetch('http://localhost:4000/api/tareas')  // estado inicial
      .then(r => r.json()).then(setTareas);
    socket.on('tarea:creada', t => setTareas(p => [...p, t]));
    socket.on('tarea:eliminada', ({ id }) =>
      setTareas(p => p.filter(x => x.id !== id)));
    return () => {              // limpieza al desmontar
      socket.off('tarea:creada');
      socket.off('tarea:eliminada');
    };
  }, []);
  return tareas;
}
```

```
$ npm i socket.io-client
```

Estado inmutable

setTareas(p => ...) evita perder eventos que llegan juntos

socket.off()

sin limpieza, cada render duplica listeners

Prueba

abre dos pestañas: lo que crees en una aparece en la otra

EJEMPLO 4 · SEGURIDAD DEL HANDSHAKE

Solo se conecta quien presenta un token válido

```
// ej4/server.js
const TOKENS_VALIDOS = { abc123: 'Ana' };

io.use((socket, next) => {  // middleware
  const token = socket.handshake.auth.token;
  // próxima clase: jwt.verify(token, SECRETO)
  const usuario = TOKENS_VALIDOS[token];
  if (!usuario) return next(new Error('No autorizado'));
  socket.data.usuario = usuario;
  next();
});

io.on('connection', (s) =>
  s.emit('bienvenida', `Hola ${s.data.usuario}`));
```

```
// cliente
io(URL, {
  auth: { token: 'abc123' }
});
```

salida real

```
[server] Ana autenticado
[cliente OK] Hola Ana
[cliente sin token]
rechazado: No autorizado
```

CRITERIO DE INGENIERO

WebSocket no es para todo: también tiene costos

Mejor otra opción cuando...

- los datos cambian cada horas: REST con caché basta
- solo el servidor envía (avisos, progreso): SSE es más simple
- es un formulario CRUD clásico sin colaboración
- corres en funciones serverless de vida corta

Lo que cuesta mantenerlo

- cada conexión abierta ocupa memoria en el servidor
- el servidor guarda estado: escalar a varias instancias es más difícil
- no aprovecha la caché HTTP ni las CDN
- hay que manejar reconexión, latidos y reintentos

Pregunta guía: **¿el usuario pierde algo si el dato le llega 10 segundos tarde?** Si la respuesta es no, quizá no necesitas tiempo real.

ANTES DE PRODUCCIÓN

Seis buenas prácticas

Usa wss://
cifrado TLS, igual que https

Autentica en el handshake
rechaza antes de abrir el canal

Valida cada mensaje
lo que llega del cliente es entrada no confiable

Usa salas
no envíes todo a todos: privacidad y ancho de banda

Reconexión y heartbeat
ping/pong detecta conexiones caídas

Escala con un adapter
varias instancias: Redis adapter + sticky sessions

ANTES DEL LABORATORIO

Cinco errores que vas a encontrar (y cómo salir)

Síntoma	Causa	Solución
Error de CORS al conectar	El origen del front-end no está permitido	new Server(server, { cors: { origin: 'http://localhost:5173' } })
La API responde pero el socket no conecta	app.listen() creó un segundo servidor	Crear http.Server y llamar server.listen()
Cada mensaje aparece dos o tres veces	Listeners registrados en cada render	socket.off() en la limpieza del useEffect
Un cliente ws no logra hablar con Socket.IO	Son protocolos distintos	Usar socket.io-client en el front-end
Funciona en local y falla en producción	ws:// desde una página https (contenido mixto)	Usar wss:// y un proxy que permita Upgrade

PRÁCTICA · 22 MINUTOS · EN PAREJAS

Laboratorio: el Slack de la clase

Un chat con canales al que toda la sección se conecta desde su laptop o celular. El front-end y las rutas REST ya están listos: tú integras Socket.IO en el back-end.

Pasos

1. Abre slack-clase-inicial y ejecuta npm install
2. TODO 1–2: entrar con un nombre y unirse a un canal (salas)
3. TODO 3: guardar el mensaje y después emitir mensaje:nuevo a la sala
4. TODO 4–6: “está escribiendo…”, canal creado y usuarios en línea
5. Prueba con dos pestañas y luego conéctense todos al servidor del profe

Reto extra

Mensajes directos (una sala privada por par de usuarios) o reacciones con emoji en tiempo real

Entregable

Repositorio + GIF de 30 s con al menos 3 compañeros chateando en tu servidor

LABORATORIO · EL CONTRATO

Eventos y rutas que tu servidor debe cumplir

Tipo	Nombre	Dirección	Datos
WS	usuario:entrar	cliente → servidor	nombre, ack
WS	canal:unirse	cliente → servidor	canal
WS	mensaje:enviar	cliente → servidor	{ canal, texto }, ack
WS	mensaje:nuevo	servidor → sala	{ canal, de, texto, hora }
WS	escribiendo	sala, menos el emisor	nombre
WS	usuarios · canal:creado	servidor → todos	lista de nombres · canal
REST	GET /api/canales	listar canales	200 [“general”, ...]
REST	GET /api/canales/:canal/mensajes	historial al abrir	200 [mensajes] · 404
REST	POST /api/canales	crear canal	201 · 400 · 409

LABORATORIO · TODOS AL MISMO CHAT

¿Cómo entramos todos al mismo Slack?

01

Un servidor

El profe (o un grupo voluntario) ejecuta node server.js

02

Abierto a la red

Escucha en 0.0.0.0 e imprime su IP, por ejemplo 192.168.1.25:3000

03

Misma Wi-Fi

Todos abren esa URL en el navegador de su laptop o celular

04

Firewall

Permitir el puerto 3000; Windows lo pregunta la primera vez

```
// '0.0.0.0' = acepta conexiones de otros equipos
server.listen(3000, '0.0.0.0');

$ node slack-clase/server.js
Slack de la clase en http://localhost:3000
Compañeros:  http://192.168.1.25:3000
```

Plan B

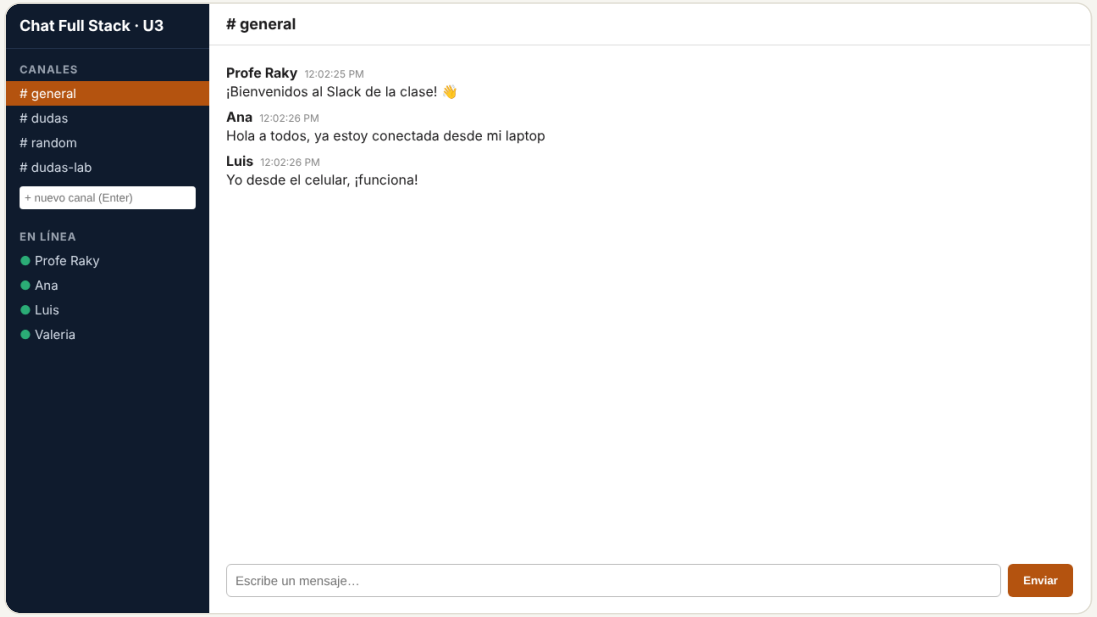
Si la red de la universidad aísla los equipos: compartir el hotspot del celular del profe, o publicar el servidor en un hosting que soporte WebSockets

LABORATORIO · RESULTADO ESPERADO

Cuatro navegadores, un solo servidor



Vista del profe en **#dudas-lab**: Valeria está escribiendo



Vista de Luis en **#general**, al mismo tiempo

REST: historial al abrir un canal; #dudas-lab se creó con POST

WebSocket: mensajes, “escribiendo…” y usuarios en línea

Salas: lo de #dudas-lab no aparece en #general

EVALUACIÓN DEL LABORATORIO · 20 PUNTOS

Rúbrica del Slack de la clase

Criterio	Logrado · 4 pts	En proceso · 2 pts
Usuarios y salas	Entra con nombre, la lista “en línea” se actualiza y el cambio de canal usa join/leave	La lista no se actualiza al salir
Mensajes en tiempo real	Valida, guarda y después emite solo a la sala correcta, con ack	Llegan a todos los canales o sin validar
REST + historial	El historial carga al abrir un canal y los canales nuevos aparecen en todos	Falla el historial o el canal nuevo
Experiencia en vivo	“Está escribiendo…” llega a los demás, no al emisor	El aviso también llega al emisor
Prueba con la clase	3 o más compañeros conectados desde otros equipos; GIF y repositorio	Solo probado en local

No logrado = 0 pts por criterio. El reto extra suma hasta 2 pts de bonificación (máximo 20).

CIERRE · TICKET DE SALIDA

Responde en una frase cada pregunta

1

¿Qué código HTTP confirma que la conexión pasó a WebSocket y qué significa?

2

¿Por qué la API REST sigue escribiendo en la BD y el WebSocket solo notifica?

3

¿Qué parte del laboratorio te costó más y cómo la resolviste?

PRÓXIMA SESIÓN

Autenticación y autorización con JSON Web Token

TOKENS_VALIDOS[token] → jwt.verify(token, SECRETO)

Para repasar: RFC 6455 (protocolo WebSocket) · socket.io/docs/v4 · github.com/websockets/ws